

Лабораторная работа №8

Вёрстка

Цель: рассмотрение способов использования каскадных таблиц стилей при оформлении web-страниц, областей применения CSS.

Задачи:

- 1) изучить виды селекторов, их свойств и значения этих свойств;
- 2) использовать селекторы, их свойства и значения этих свойств для оформления web-страниц.

Теоретические сведения

Верстка веб-страниц представляет собой создание гипертекстового документа на основе HTML-разметки при использовании таблиц стилей, таким образом, чтобы элементы дизайна выглядели аналогично макету. Верстка веб-страниц отличается от полиграфической тем, что необходимо учитывать разницу отображения элементов в различных браузерах и разницу в размерах рабочего пространства устройств.

В **HTML 5** для каждой части страницы имеется свой элемент.

Вот их список и краткое назначение:

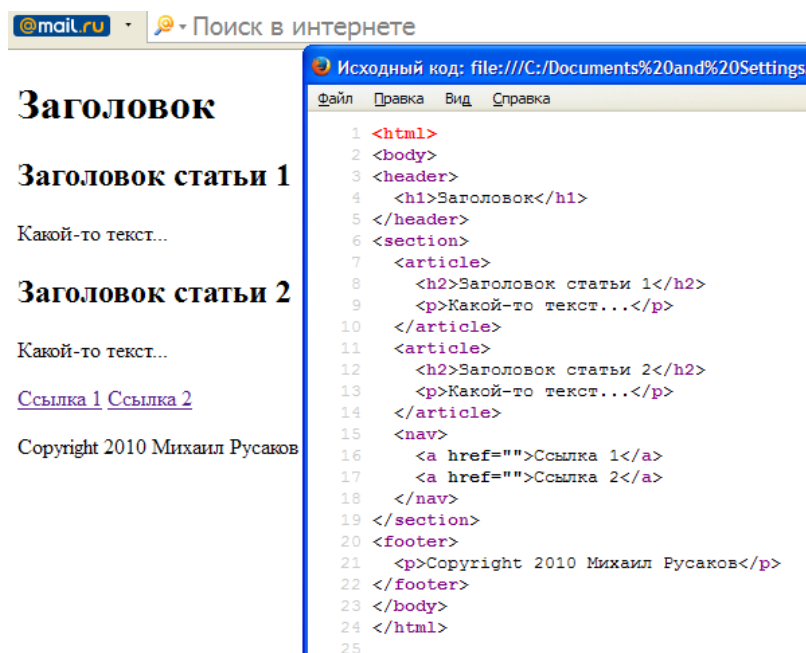
- 1) **section**. Назначение - определение секций. Его используют для описания определённого блока текста, например, хорошим применением этого элемента будет при разбиении большой части текста на более малые, как разбиение одной статьи на несколько абзацев.
- 2) **header**. Назначение - определение верхней секции на странице.
- 3) **footer**. Назначение - определение нижней секции на странице.
- 4) **nav**. Назначение - определяет набор ссылок на другие страницы (часто используют для навигации по сайту).
- 5) **article**. Назначение - выделить определённую часть текста.

Тег	Описание
<article> (статья)	Определяет независимый, самодостаточный контент: это может быть новостная статья, пост блога, форума или другие статьи, которые могут распространяться независимо от остального содержимого
<aside> (отступление)	Для контента - отступления от темы, в которой он размещен. Отступление должно быть связано с окружающим контентом
<command> (команда)	Кнопка, или переключатель (радио кнопка), или флажок
<details> (детали)	Для описания подробностей документа или частей документа
<summary> (резюме)	Заголовок (краткое содержание, резюме) внутри элемента details

<figure> (иллюстрация, рисунок)	Для группировки отдельно стоящего содержимого, может быть видео
<figcaption> (заголовок)	Заголовок раздела иллюстрации, рисунка, схемы и т.п.
<footer> (нижний колонтитул, подвал)	Для создания нижнего колонтитула документа или раздела, может включать, например, имя автора, число, когда документ был опубликован, контактную информацию, информацию об авторском праве
<header> (заголовок)	Для создания вступительной части документа или раздела, может включать панель навигации
<hgroup>	Для группы заголовков с использованием <h1> - <h6>, где наибольший - это главный заголовок раздела, а остальные - это подзаголовки
<mark>	Для текста, который необходимо выделить
<meter>	Для измерения, используется только если максимальное и минимальное значения известны
<nav>	Для панели навигации
<progress>	Прогресс состояния работы
<rt>	Для объяснения ruby-комментария
<rp>	Что показывать браузерам, которые не поддерживают элемент ruby
<section>	Для раздела (колонки, рубрики) в документе. Используется для создания глав, заголовков, нижних колонтитулов или других разделов документа
<time> (время)	Для определения времени или даты, или того и другого
<wbr>	Разделение слов. Для определения возможности разделения слова с целью переноса на другую строку посредством дефиса.

Пример. Использование элементов разметки HTML5

В примере ниже начала определили заголовок в теге **<header>**. Затем добавили секцию с помощью тега **<section>**, внутри которого написали два тега **<article>**, которые выделяют определённую часть текста. После секции поставили набор ссылок с помощью тега **<nav>**.



Гиперссылки

Для создания ссылки необходимо сообщить браузеру, что является ссылкой, а также указать адрес документа, на который следует сделать ссылку. Оба действия выполняются с помощью тега `A`, который имеет единственный параметр `href`. В качестве значения используется адрес документа (URL).

Абсолютные ссылки используются в том случае, если мы хотим перейти на документ, адрес которого неизменен. Это, прежде всего, ссылки на внешние интернет-ресурсы, например, данный фрагмент

`<a href="http://www.ya.ru">Спросить у Яндекса</a>`

Для внутренних ресурсов необходимо использовать **относительные ссылки**. В них дорога к искомому документу считается относительно от того места, где вы сейчас находитесь, например:

`<a href="../index.htm">в начало</a>`.

Внутренние ссылки используются для быстрого перемещения внутри документа. Для таких перемещений, во-первых, нужно создать внутри html-документа

«именованные якоря».



Схема внутренних ссылок.

Они помечаются следующим фрагментом кода: `<a name="1"></a>`.

Чтобы совершить переход в произвольном месте страницы, нужно добавить следующую ссылку: `<a href="#1">ссылка</a>`. Знак решетка, сообщит браузеру, что переход совершается внутри страницы. Можно, также, делать ссылку на закладку, находящуюся в другой веб-странице и даже другом сайте. Для этого в адресе ссылки надлежит указать ее адрес и в конце добавить символ решетки # и имя закладки.

Пример. Вставка изображения в документ

```
<html>
<body>
 - это абсолютный адрес размещения
изображения
 - адрес размещения изображения относительно корня сайта
 - адрес размещения изображения относительно текущего
HTML-документа
</body>
</html>
```

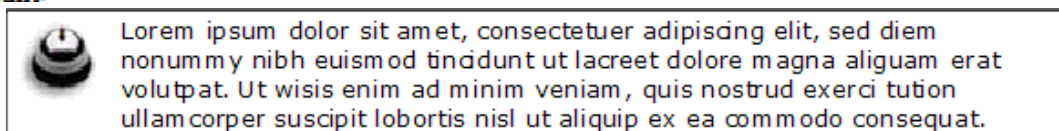
Выравнивание изображений

Способ выравнивания изображений задается параметром **align** тега **IMG**.

Наиболее популярные параметры - **left** и **right**, создающие обтекание текста вокруг изображения. Чтобы текст не прилегал плотно к рисунку, рекомендуется в теге **IMG** добавить параметр **hspace** и **vspace**, задающих расстояние до текста в пикселах.

Пример. Обтекание текста вокруг рисунка

```
<html>
<body>
Lorem ipsum
dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet
dolore magna aliquam erat volutpat. Ut wisis enim ad minim veniam, quis nostrud exerci tution
ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat.
</body>
</html>
```



Создание карт-изображений

Карты - это способ сделать различные части одного графического изображения гиперссылками. Они позволяют выделить отдельные области изображений и определить для каждой из них свое действие.

1. Чтобы создать карту нужно вставить в тег `<IMG SRC="">` атрибут `USEMAP="#name"`, где **name** - имя карты (значок # обязателен).
2. Далее описываются активные области карты. Начините с открывающегося тега `<MAP NAME="name">` (здесь повторяется имя, но уже без значка #), а заканчивайте закрывающимся тегом `</MAP>`.
3. Между этими тегами поместите описание каждой активной области изображения:
`<AREA SHAPE="форма" COORDS="координаты" HREF="адрес" TITLE="альтернативный текст">`.
Элемент `<AREA>` имеет следующие атрибуты и их значения:

SHAPE	Описывает форму выделяемой области, возможные значения: RECT - прямоугольник CIRCLE - круг POLY - многоугольник DEFAULT - определяет всю область, т.е. весь рисунок может стать ссылкой.
COORDS	Координаты, определяющие размеры и положение области на изображении. Все координаты отсчитываются в пикселях от левого верхнего угла изображения. Количество и порядок значений зависит от значения атрибута SHAPE: RECT: - левый-X, верхний-Y, правый-X, нижний-Y (т.е. сначала координаты левого верхнего угла, затем правого нижнего) CIRCLE: - центр-X, центр-Y, радиус (т.е. горизонтальная и вертикальная координаты центра круга и радиус) POLY: - X1, Y1, X2, Y2, ..., Xn, Yn (просто перечисляются координаты всех вершин многоугольника).
ALT	Альтернативный текст для выделенной области, используется невидимыми браузерами.
TITLE	Название выделенной области, выводится в виде подсказки, всплывающей при наведении курсора на область рисунка.
TARGET	Значение этого атрибута ("_top", "_blank", "_self" или "_parent") определяет, в каком окне будет открыт документ

Пример создания карты-изображения

```

<map name="mymap">
  <area shape="rect" coords="0,0,82,126" alt="участок карты 1" href="1.htm" />
  <area shape="circle" coords="90,58,3" alt="участок карты 2" href="2.htm" />
  <area shape="circle" coords="124,58,8" alt="участок карты 3" href="3.htm" />
</map>
```

Абсолютное позиционирование элемента

Обычно элемент веб-страницы отображается после предыдущих элементов, записанных в коде HTML-документа. Рассмотрим примеры.

<code><p>Текст абзаца </p></code>	Текст абзаца ➡
---	----------------

Тег `<img>` в документе записан после текста абзаца, значит и на странице картинка отобразится после текста.

```
<p>Воздушный шарик состоит из:</p>
<ul>
<li>резиновой оболочки;</li>
<li>завязки;</li>
<li>воздуха.</li>
</ul>
```

Воздушный шарик состоит из:

- резиновой оболочки;
- завязки;
- воздуха.

Код `<ul>` записан после `<p>`, поэтому на странице список отобразится ниже параграфа.

Таким образом, элементы на странице по умолчанию никогда не перекрываются. Однако этот порядок можно менять с помощью CSS-свойств `position`, `float` и `clear`.

Position

CSS-свойство `position` позиционирует элемент независимо от расположения предшествующих элементов. Свойство имеет два важных значения: `absolute` и `relative`.

Элемент со свойством `position: absolute` располагается независимо от всех других элементов. Координаты этого элемента относительно левого и верхнего краёв страницы определяются свойствами `left` и `top`.

Приведём код CSS-правила для букета, который расположился в левом верхнем углу страницы:

```
position: absolute; left:
0px;
top: 0px;
```

Значение `absolute` удобно в том случае, когда элемент нужно “привязать” к углу или краю страницы. При редактировании страницы, добавлении и удалении содержимого, элемент будет оставаться на месте независимо от других.

Плавающий элемент

Так как тег `<p>` является блочным, то он отображается ниже предыдущего элемента с новой строки. В примере ниже записаны обычные теги картинки и двух абзацев:

```

<p>Эти грациозные белоснежные птицы во все времена привлекали людей своей
строгой красотой. ... </p>
<p>Но восторг красотой и воспевание лебедей в поэзии не мешали людям охотиться
на этих крупных водоплавающих птиц ... </p>
```

Результат:



Эти грациозные белоснежные птицы во все времена привлекали людей своей строгой красотой. Лебеди с самых ранних веков человеческой истории служили в Европе символом верности и чести. Их образ являлся олицетворением женской красоты, нежности, романтики, поэзии и верности. В античности лебедь считался птицей Афродиты и Аполлона, а христианская иконография сделала его птицей Девы Марии.

Но восторг красотой и воспевание лебедей в поэзии не мешали людям охотиться на этих крупных водоплавающих птиц ради мяса, пуха и пера. Так что, к настоящему времени многие виды лебедей находятся под угрозой и занесены в Красную книгу. Сохранению поголовья лебедей помогают не только различные природоохранные мероприятия, но и, ставшее в последнее время распространённым, разведение лебедей в неволе.

Справа от картинки остаётся пустое место, что не красиво. Кроме того неэффективно используется площадь страницы. Необходимо, чтобы текст

обтекалка картинку справа.

CSS-свойство **float** (плавать) прижимает элемент к краю страницы, чтобы остальные могли его «обтекать» с другой стороны. Свойство **float** имеет два важных значения:

- **left** сдвигает элемент к левому краю страницы;
- **right** сдвигает элемент к правому краю.

Пример:

```
.обтекание {
  float: left;
  margin-right: 15px;
}


<p>Эти грациозные белоснежные птицы во все времена привлекали людей своей
строгой красотой. ... </p>
<p>Но восторг красотой и воспевание лебедей в поэзии не мешали людям охотиться
на этих крупных водоплавающих птиц ... </p>
```

Результат показан на рис. 3.



Эти грациозные белоснежные птицы во все времена привлекали людей своей строгой красотой. Лебеди с самых ранних веков человеческой истории служили в Европе символом весны и тепла. Их образ являлся олицетворением женской красоты, нежности, романтики, поэзии и верности. В античности лебедь считался птицей Афродиты и Аполлона, а христианская иконография сделала его птицей Девы Марии.

Но восторг красотой и воспевание лебедей в поэзии не мешали людям охотиться на этих крупных водоплавающих птиц ради мяса, пуха и пера. Так что, к настоящему времени многие виды лебедей находятся под угрозой и занесены в Красную книгу. Сохранению поголовья лебедей помогают на только различные природоохранные мероприятия, но и, ставшее в последнее время распространённым, разведение лебедей в неволе.

Когда требуется, чтобы второй абзац расположился ниже картинки, то необходимо «досрочное» прекращение обтекания.

CSS-свойство **clear** (очищать, освобождать) освобождает от обтекания предыдущих плавающих элементов. Чаще всего используют значение **both**, которое освобождает от обтекания как слева, так и справа. В примере ниже второму абзацуприсвоено свойство **clear:both**.

```
.обтекание {
  float: left;
  margin-right: 15px;
}
.ниже {
  clear: both;
}


<p>Эти грациозные белоснежные птицы во все времена привлекали людей своей
строгой красотой. ... </p>
<p class="ниже">Но восторг красотой и воспевание лебедей в поэзии не мешали
людям охотиться на этих крупных водоплавающих птиц ... </p>
```

Результат:



Эти грациозные белоснежные птицы во все времена привлекали людей своей строгой красотой. Лебеди с самых ранних веков человеческой истории служили в Европе символом весны и тепла. Их образ являлся олицетворением женской красоты, нежности, романтики, поэзии и верности. В античности лебедь считался птицей Афродиты и Аполлона, а христианская иконография сделала его птицей Девы Марии.

Но восторг красотой и воспевание лебедей в поэзии не мешали людям охотиться на этих крупных водоплавающих птиц ради мяса, пуха и пера. Так что, к настоящему времени многие виды лебедей находятся под угрозой и занесены в Красную книгу. Сохранению поголовья лебедей помогают на только различные природоохранные мероприятия, но и, ставшее в последнее время распространённым, разведение лебедей в неволе.

position	позиционирование элемента
left	сдвиг относительно левой границы

top	сдвиг относительно верхней границы
float	плавающий элемент
clear	освобождение от плавающего элемента

Простой шаблон блочной верстки

Представим необходимую структуру графически:



После того как мы нарисовали структуру в графическом варианте мы можем определиться с тем какие теги мы будем использовать для создания структуры страницы с использованием HTML5:

```

FOLDERS
  ПР05Блоки
    index.html
    style.css

index.html
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <title>ПР05 Блочная верстка CSS</title>
6  </head>
7  <body>
8      <header> Шапка сайта
9      </header>
10     <nav>
11         Меню навигации горизонтальное
12     </nav>
13     <aside>
14         Левый сайдбар
15     </aside>
16     <div>
17         Основная часть
18     </div>
19     <div></div>
20     <footer> Подвал
21     </footer>
22 </body>
23 </html>
24

```

Если мы посмотрим как выглядит наш код в браузере то увидим следующее

Шапка сайта
Меню навигации горизонтальное
Левый сайдбар
Основная часть
Подвал

Добавляем стили CSS в часть <head>

```
1  <style> body {
2    background: #f5f5f5;
3    color: #000000;
4    font-family: Arial, Times New Roman;
5    font-size: 16px;
6  }
7  header {
8    background: yellow
9    height: 100px;
10   width: 100%;
11 }
12 nav {
13   background: #330044; color: white;
14   width: 100%; height: 50px;
15 }
16 aside {
17   background: #1baf5d; float: left;
18   width: 10%;
19
20   height: 400px;
21 }
22 #content { background: gray; float: left;
23 width: 90%;
24 }
25 #clear { clear: both;
26
27 }
28 footer {
29   background: #ff0404; color: white;
30   height: 80px; width: 100%;
31 }
32 </style>
```

Проверим результат в браузере:



После применения стилей видно что наш сайт уже стал приобретать вид того что нам нужно. Теперь разберем код стилей.

Итак, начнем по порядку с тега `<header>` нам необходимо задать ему ширину в 100% от окна, это делается с помощью свойства `width:100%`, и задаем высоту в 100 пикселей с помощью свойства `height:100px`. И соответственно используя свойство `background: yellow`. Шапка готова. Значения ширины и высоты могут принимать значения в пикселях и в процентах. Что использовать зависит от конкретных ситуаций. Переходим дальше. Следующим элементом у нас является меню навигации. При стилизации блока меню используются те же самые свойства что и при стилизации шапки. Плюс мы добавили свойство `color: white`, оно позволяет указать цвет текста. В качестве значений для свойств задающих цвета могут выступать названия цветов (например, `red`, `Green`, `black`), так и шестнадцатичные значения цветов (например, `#ff00ff`, `#000000` и т.д.).

Следующим блоком у нас является сайдбар созданный с помощью тега `<aside>`, к нему применяем свойства ширины, высоты, цвет заливки и добавляется свойство `float:left`, оно позволяет задать позиционирование блока по левому краю. При задании размеров блоков нужно правильно рассчитывать ширину блоков, которые находятся рядом. Вся ширина экрана это 100%, то есть для того чтобы нам разместить два блока рядом необходимо одному блоку задать ширину X второму задать ширину Y , но главное следует всегда просчитывать чтобы сумма значений X и Y всегда равнялась 100%. В противном случае может появиться пустое пространство, либо блоки не будут уместиться и будут не структурированы, то есть какой либо из блоков может переместиться ниже или выше другого, либо наложиться сверху.

Следующий блок это основная часть страницы, его мы создали с помощью тега `<div>` и присвоили ему идентификатор `#content`. данному идентификатору мы создали стили заливки фона (`background`), ширину (`width`), высоту (`height`), и обтекаемость (`float`).

Последним блоком является подвал, который мы создали с помощью тега `<footer>`. Стили для данного блока практически идентичны стилям которые мы использовали при стилизации шапки страницы.

Это небольшой пример того как можно сверстать веб-страницу с помощью блоков и стилей CSS. Большие и качественные проекты безусловно требуют гораздо более сложного кода и структуры тегов. Но всегда нужно с чего-то начинать и данная работа поможет вам в этом.

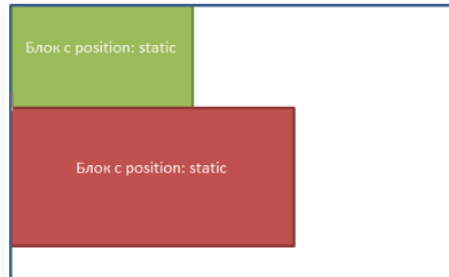
Примечание: В случае отличий расположения блоков с образцом, выполните позиционирование. Используйте, например, для подвала свойство `position: absolute`; или `position: fixed`;

Свойство `position` со значением `static` элементам назначается по умолчанию. Это значение означает что элемент является не позиционированным, т.е. отображается как обычно (в потоке). Явная установка элементу CSS-свойства `position: static` может понадобиться только в том случае, когда нужно переопределить другое значение `position` установленное элементу. Установка CSS свойств для задания положения элемента `left`, `top`, `right` и `bottom` никакого влияния на него не оказывают, т.к. его местонахождение определяется потоком документа. Пример выстраивания статично позиционированных элементов:

```

<body>
<div style="width: 200px; height: 100px; border: 1px solid black; background:
green;"></div>
<div style="width: 300px; height: 150px; border: 1px solid black; background:
red;"></div>
</body>

```



Установка относительного позиционирования элементу осуществляется посредством задания ему CSS свойства `position: relative`. Относительно позиционированный элемент ведёт себя как элемент в потоке за исключением того, что его текущее положение можно при помощи определённых CSS свойств сместить. К этим CSS свойствам относятся `left`, `top`, `right` и `bottom`. Например, для того чтобы элемент сдвинуть вверх или вниз относительно его исходного положения к нему нужно применить CSS свойство `top` или `bottom`:

```

position: relative;
/* для сдвига элемента вверх на 10px */
top: -10px; /* или bottom: 10px; */

```

```

/* для сдвига элемента вниз на 10px */
top: 10px; /* или bottom: -10px; */

```

Если одновременно установить `top` и `bottom`, то будет применено значение `top`, т.к. оно является более приоритетным, чем `bottom`:

```

position: relative;
/* элемент или элементы, к которым применяется эти стили будут сдвинуты на
15px вверх, а не на 10px как указано в bottom */
top: -15px;
bottom: 10px;

```

Для сдвига элемента вправо или влево используется CSS свойство `left` или `right`:

```

position: relative;
/* для сдвига элемента влево на 20px */
left: -20px; /* или right: 20px; */
/* для сдвига элемента вправо на 20px */
left: 20px; /* или right: -20px; */

```

Если одновременно установить `left` и `right`, то приоритетным будет значение, находящееся в `left`:

```

position: relative;
/* элемент или элементы, к которым применяется эти стили будут сдвинуты на

```

25px вправо, т.к. значение left более приоритетно чем right */

left: 25px; right: -

20px;

Для сдвига по двум осям нужно использовать top или bottom, и left или right:

position: relative;

/* стили для сдвига элементов вверх и влево на 5px

*/top: -5px;

left: -5px;

Пример, в котором 2 элементу установим относительное позиционирование и сместим его на 20px вверх и влево относительно его исходного положения:

```
<div class="container">
```

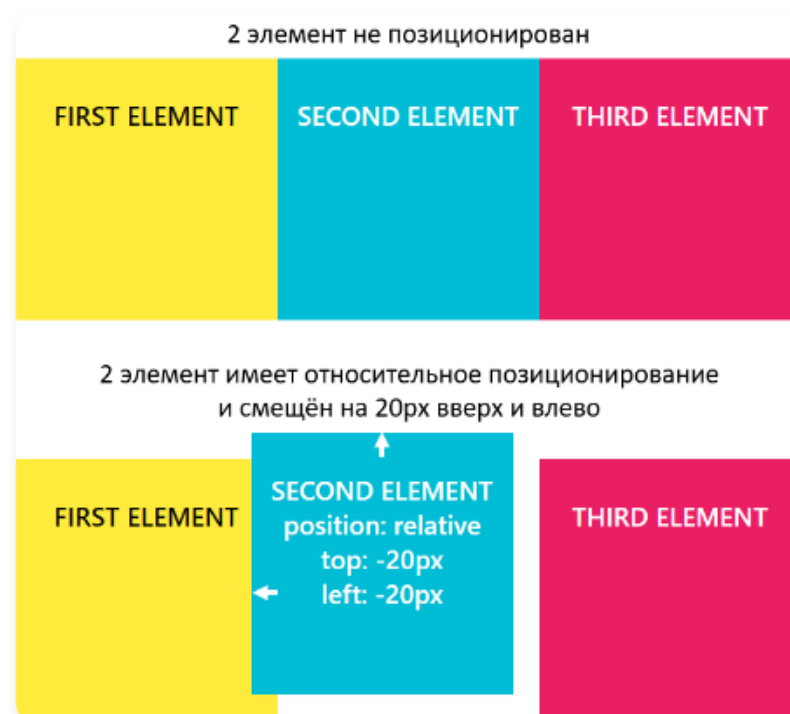
```
  <div class="element-1">FIRST ELEMENT</div>
```

```
  <!-- элемент имеет относительное позиционирование и смещён на 20px  
  вверх и влево -->
```

```
    <div class="element-2" style="position: relative; top: -20px; left: -20px;">SECOND  
    ELEMENT</div>
```

```
    <div class="element-3">THIRD ELEMENT</div>
```

```
</div>
```



Если в некоторой области страницы оказываются несколько позиционированных элементов, то они перекрывают друг на друга в определённом порядке. При этом по умолчанию выше оказывается тот элемент, который ниже описан в коде. Но порядок перекрывания элементов (их положение перпендикулярное экрану, т.е. вдоль оси Z) можно изменить. Осуществляется в CSS это с помощью свойства z-index. z-

index может принимать отрицательные и положительные целые число, auto и 0. Но, хорошей практикой является использование в качестве z-index чисел из диапазона 0-9999.

При этом чем больше у элемента значение z-index, тем ближе он располагается к нам, и, следовательно, перекрывает все элементы в данной области, у которых значение z-index меньше.

Установка абсолютного позиционирования элементу осуществляется посредством задания ему position: absolute. Этот тип позиционирования позволяет разместить элемент именно там, где вы хотите. Позиционирование выполняется относительно ближайшего позиционированного предка.

```
<div id="id-1" style="position: absolute">
  <div id="id-2" style="position: relative">
    <div id="id-3" style="position: absolute">
      ...
    </div>
  </div>
</div>
```

Под позиционированным элементом понимается элемент с position, равным relative, absolute, fixed или sticky.

В этом примере позиционирование элемента #id-3 будет выполнять относительно #id-2, т.к. он является позиционированным и является по отношению к нему более близким предком. Если данный элемент не был бы позиционированным, то позиционирование #id-3 выполнялось бы относительно #id-1:

```
<div id="id-1" style="position: absolute">
  <div id="id-2">
    <div id="id-3" style="position: absolute">
      ...
    </div>
  </div>
</div>
```

Если среди предков у элемента с position: absolute нет позиционированного элемента, то в этом случае он будет позиционироваться относительно HTML страницы, т.е. элемента body. Когда элементу устанавливаем position: absolute без указания CSS-свойств (top, left, right и bottom), определяющих его положение, он будет находиться в том месте, в котором он был бы расположен, если бы находился в потоке (при этом при вычислении его положения учитываются только элементы, расположенные до него в коде и находящиеся в потоке). При этом другие элементы его видеть не будут, и, следовательно, они будут расположены на странице, не обращая никакого внимания на него.

```
<div class="container">
```

```
  <div class="element-1">FIRST ELEMENT</div>
```

```
  <!-- элемент имеет абсолютное позиционирование и ему не установлены CSS-  
свойстватop, bottom, left и right -->
```

```
    <div class="element-2" style="position: absolute;">SECOND ELEMENT</div>
```

```
    <div class="element-3">THIRD ELEMENT</div>
```

```
</div>
```



CSS-свойства для управления положением абсолютно позиционированного элемента работают по-другому чем с `position: relative`. CSS-свойства `top`, `bottom`, `left` и `right` задают положение элемента относительно ближайшего позиционированного предка или `body`, если такого предка нет. Установить ширину (высоту) абсолютно позиционированному можно с помощью установки ему двух координат `top` и `bottom` (`left` и `right`). Если элементу одновременно установить `top`, `bottom` и `height`, то предпочтение будет отдано `top` и `height`.

Абсолютное позиционирование применяется очень часто совместно с относительным позиционированием в дизайнерских целях, когда необходимо разместить различные элементы относительно друг друга, так же может применяться для создания выпадающих меню, разметки сайта и т.д.

```
<body>
```

```
  <div style="width: 200px; height: 100px; border: 1px solid black; background:  
green;"></div>
```

```
  <div style="width: 300px; height: 200px; position: absolute; top: 50px; left: 100px;  
border: 1px solid black; background:red;"></div>
```

```
</body>
```




Фиксированное позиционирование (`position: fixed`) похоже на абсолютное, но в отличие от него оно всегда привязывается к краям окна браузера (viewport), и остаётся в таком положении даже при скроллинге страницы. Фиксированное позиционирование применяется для закрепления на странице навигационных меню, кнопки «вверх», панелей с социальными кнопками и т.п.

```
<body>
  <div style="width: 200px; height: 2000px; border: 1px solid black; background:
green;"></div>
  <div style="width: 600px; height: 200px; position: fixed; top: 100px; left: 100px;
border: 1px solid black; background: red;"></div>
</body>
```



Относительное позиционирование очень часто используется вместе с абсолютным позиционированием.

Рассмотрим варианты:

Если расположить блоки с абсолютным позиционированием в блок с относительным, то расстояния будут уже задаваться не от края окна браузера, а от границ относительного блока:

```
<body>
```

```
<!-- Зелёный блок с относительным позиционированием -->
```

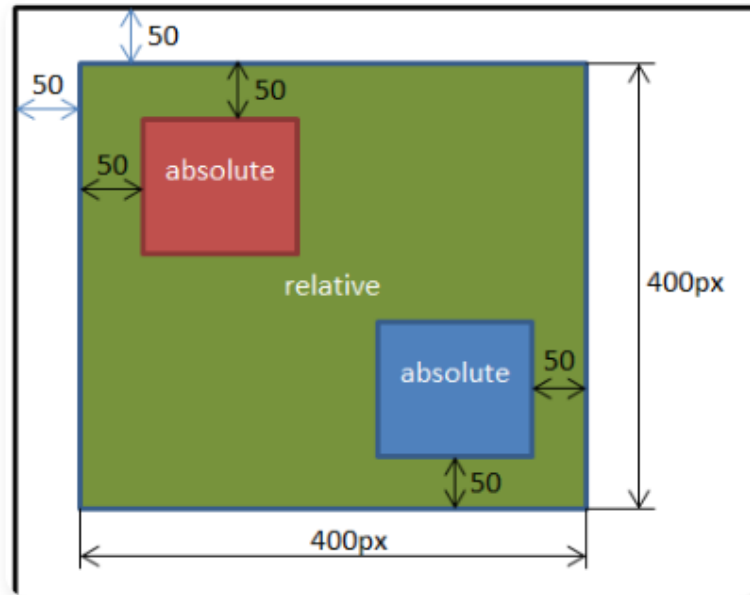
```
<div style="width: 400px; height: 400px; position: relative; top: 50px; left: 50px; border:
1px solid black; background: green;">
```

<!-- Красный блок с абсолютным позиционированием -->

```
<div style="width: 100px; height:100px; position:absolute; top:50px; left:50px; border: 1pxsolid black; background:red;"></div>
```

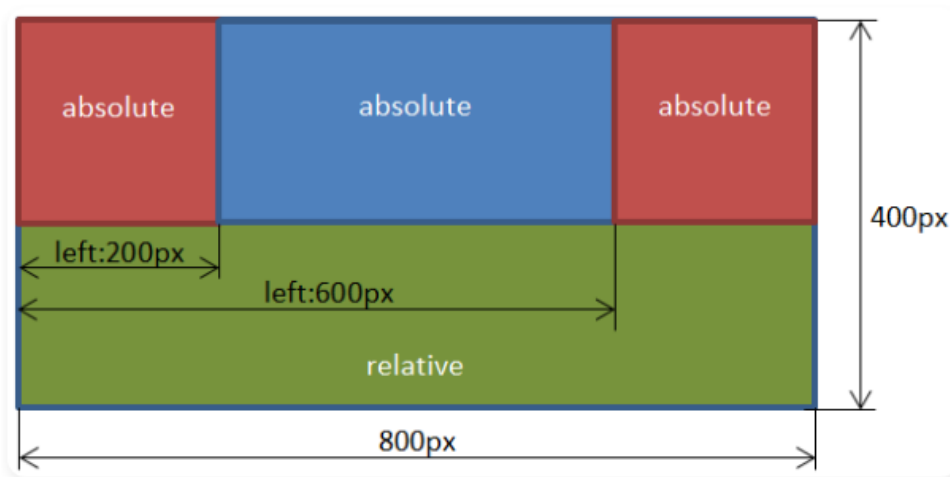
<!-- Синий блок с абсолютным позиционированием -->

```
<div style="width: 100px; height:100px; position:absolute; bottom:50px; right:50px; border:1px solid black; background:blue;"></div>
</div>
</body>
```



Например: для создания фиксированных макетов состоящих из 3 блоков, выровненных по верхнему краю. Установим высоту "400px" относительно блоку для наглядности:

```
<body>
<div style="width: 800px; height:400px; position:relative; border:
  1px solid black; background:green;">
<div style="width: 200px; height:200px; position:absolute; left:0px; border: 1px solid
black; background:red;">Левый блок</div>
<div style="width: 400px; height:200px; position:absolute; left:200px; border: 1px solid
black; background:blue;">Основной блок</div>
<div style="width: 200px; height:200px; position:absolute; left:600px; border: 1px solid
black; background:red;">Правый блок</div>
</div>
</body>
```



Дополнительно к блокам можно применять свойство `z-index`, которое предназначено для позиционирования элементов по оси Z. Чем больше значение свойства `z-index`, тем ближе элемент расположен к нам, и наоборот, чем меньше значение, тем дальше расположен элемент от нас.

Практическое задание:

Сделать верстку веб-страниц по собственному дизайн-макету.

Время выполнения работы: 9 часов.